UNIVERSITATEA NAȚIONALĂ de ȘTIINȚĂ și TEHNOLOGIE POLITEHNICA din BUCUREŞTI

Facultatea de Electronică, Telecomunicaţii şi Tehnologia Informaţiei

Proiect

Arhitectura **M**icro**p**rocesoarelor **2**. **M**icro**c**ontrolere

Sistem de Monitorizare și Avertizare pentru Cutremure

Necula Adrian-Nicușor

Grupa 422E

București 2025

Cuprins

Lista figurilor & Lista tabelelor..... 3

Introducere..... 4
1. Scopul proiectului
2. Contextul tehnic
3. Structura lucrării

Capitolul 1. Fundamente teoretice..... 5
1.1 Circuite Logice Combinaționale
1.2 Circuite Logice Secvențiale
1.3 Automate Finite de Stare (FSM)
1.4 Senzorii utilizați în sisteme de detecție

Capitolul 2. Prezentarea sistemului..... 6
2.1 Descriere generală
2.2 Schema bloc a sistemului

Capitolul 3. Proiectarea și implementarea sistemului
3.1 Circuitul logic combinațional (CLC) 7
3.2 Circuitul logic secvențial (CLS) 10
3.3 Procesul secvențial (PS) – FSM-ul general..... 13
3.4 Codul complet a modulului software (CLC + CLS + PS)..... 14

Capitolul 5. Testarea și validarea sistemului..... 20
5.1 Metodologia de testare
5.2 Rezultate obținute și interpretare

Concluzii și perspective..... 22

Lista figurilor

- **Figura 2.2.1** – Schema bloc a sistemului automat de monitorizare și avertizare pentru cutremure
- **Figura 3.2.1** – Graf asociat CLS
- **Figura 3.3.1** – Graful de tranziții PS
- **Figuri 5.2** – Testare funcționare

Lista tabelelor

- **Tabel 3.1.1** – intrări CLC
- **Tabel 3.1.2** – ieșiri CLC
- **Tabel 3.1.3** – Mască VIB_R
- **Tabel 3.1.4** – Mască VIB_M
- **Tabel 3.1.5** – Mască VIB_S
- **Tabel 3.1.6** – Rezultatul shift-ării
- **Tabel 3.1.7** – INDEX
- **Tabel 3.1.8** – Semnificație combinații posibile de biți
- **Tabel 3.1.9** – Intrări / Ieșiri CLC și semnificații
- **Tabel 3.2.1** – Adresele asociate stărilor
- **Tabel 3.2.2** – Tabel de tranziție al stărilor

Introducere

1. Scopul proiectului

Scopul acestui proiect este realizarea unui sistem electronic care să detecteze cutremurele și să îi avertizeze rapid pe cei din jur. Practic, vrem să construim un circuit care folosește un senzor pentru a simți dacă pământul vibrează mai tare decât normal. Dacă apar astfel de vibrații, sistemul aprinde LED-uri de diferite culori, iar dacă pericolul este mare, pornește și un buzzer pentru a fi clar că trebuie să reacționăm.

Proiectul are și o funcție care verifică dacă vibrațiile mari țin cel puțin 3 secunde, pentru a nu da alarme false la orice mică mișcare sau zgomot. Dacă totuși vibrațiile rămân mari, se aprinde un LED roșu, iar buzzerul începe să sune. Dacă vibrațiile se opresc, sistemul revine la starea normală.

2. Contextul tehnic

Sistemul folosește un senzor (accelerometru) care detectează mișcările sau vibrațiile. Acest senzor transmite un semnal electronic către circuitul nostru, care decide cât de puternică este vibrația. Pentru asta, folosim circuite logice simple: unele decid pe loc (combinațional), iar altele „țin minte” dacă vibrațiile au durat destul (secvențiale).

Semnalele de la senzor sunt transformate în aprinderea unor LED-uri (verde, galben, roșu) în funcție de cât de mare e vibrația. Dacă sistemul detectează o vibrație mare timp de 3 secunde, pornește și un buzzer pentru a fi clar că există un pericol. Tot sistemul poate fi construit cu componente electronice de bază și poate fi controlat de un microcontroler sau de circuite logice, după nevoie.

Practic, tot ce face acest sistem e să simtă mișcările puternice și să dea imediat de știre, ca să știe oamenii că se apropie sau se întâmplă un cutremur.

3. Structura lucrării

Lucrarea este structurată astfel încât să acopere toate etapele de proiectare, implementare și testare ale sistemului propus. În proiect o să explicăm cum funcționează acest sistem, din ce componente e format (senzor, LED-uri, buzzer), cum procesează datele de la senzor, ce face fiecare parte din circuit și cum se testează ca să ne asigurăm că merge corect. Fiecare capitol prezintă pe rând schema bloc, modul în care sistemul decide dacă e vreun pericol, ce trebuie să afișeze și cum se face testarea.

Capitolul 1. Fundamente teoretice

Pentru realizarea unui sistem automat de monitorizare și avertizare la cutremure, este necesară înțelegerea conceptelor de bază din domeniul logicii digitale, al senzorilor și al sistemelor de control secvențial. În acest capitol sunt prezentate noțiunile teoretice care au stat la baza proiectului.

1.1 Circuite Logice Combinaționale

Circuitele logice combinaționale sunt sisteme digitale în care ieșirea depinde doar de valorile actuale ale intrărilor, fără memorie internă. Astfel de circuite sunt folosite pentru decizii rapide, cum ar fi determinarea nivelului de alertă în funcție de semnalul primit de la senzorul de vibrații.

În cadrul acestui proiect, circuitul logic combinațional analizează valoarea dată de senzor (nivel mic, mediu sau mare de vibrație) și aprinde LED-ul corespunzător (verde, galben, roșu). Decizia se ia instant, folosind porți logice de bază (AND, OR, NOT).

1.2 Circuite Logice Secvențiale

Circuitele logice secvențiale au capacitatea de a „ține minte” ce s-a întâmplat anterior, datorită elementelor de memorie (bistabile). Ieșirile lor depind atât de intrările curente, cât și de stările trecute.

În proiectul de față, circuitul logic secvențial verifică dacă vibrațiile mari durează cel puțin 3 secunde. Acest lucru se face cu ajutorul unui contor care se incrementează cât timp nivelul rămâne mare. Dacă timpul se scurge, sistemul pornește alarma sonoră. Dacă nu, sistemul nu declanșează alarma, reducând riscul de avertizări false.

1.3 Automate Finite de Stare (FSM)

Automatul finit de stare (FSM) este un model care ajută la controlul proceselor cu mai mulți pași, pe baza unor stări clar definite și a unor reguli de trecere între stări. În acest proiect, FSM-ul trece prin următoarele stări principale:

- Monitorizare – sistemul verifică constant nivelul de vibrație.
- Avertizare – dacă nivelul este mediu, aprinde LED galben.
- Numărătoare – dacă nivelul e ridicat, începe să contorizeze timpul.
- Alarmă activă – dacă nivelul mare persistă 3 secunde, se aprinde LED roșu și pornește buzzer-ul.
- Revenire – când vibrația scade, sistemul revine la starea de monitorizare.

Acest tip de control secvențial asigură funcționarea corectă și reacția potrivită la fiecare situație.

1.4 Senzorii utilizați în sisteme de detecție

Sistemul folosește un accelerometru, care detectează mișcările sau vibrațiile solului. Accelerometrul convertește aceste mișcări în semnale electrice, care sunt apoi interpretate de circuitul logic.

Pentru semnalizarea stărilor, sunt folosite LED-uri (pentru vizual) și un buzzer (pentru avertizare sonoră).

Capitolul 2. Prezentarea sistemului

Acest capitol detaliază procesul de proiectare a sistemului automat de monitorizare și avertizare la cutremure. Proiectarea a fost realizată în mod modular, fiecare componentă fiind gândită astfel încât să contribuie la funcționarea corectă și eficientă a întregului ansamblu. Sunt prezentate arhitectura generală, alegerea componentelor și modul în care logica digitală este folosită pentru a asigura reacția rapidă și corectă a sistemului la diferite niveluri de vibrație detectate.

2.1 Descriere generală

Sistemul automat de monitorizare și avertizare la cutremure este alcătuit dintr-un set de componente electronice care colaborează pentru a supraveghea continuu nivelul vibrațiilor din mediul ambiant. Senzorul de vibrații măsoară permanent orice mișcare a solului și transmite informațiile către un circuit digital care le interpretează rapid. Pe baza valorii detectate, sistemul aprinde automat un LED corespunzător nivelului de vibrație: verde pentru condiții normale, galben pentru atenționare și roșu pentru situații critice.

În plus, sistemul nu reacționează la orice fluctuație accidentală, ci analizează dacă o vibrație puternică persistă pentru un interval stabilit (3 secunde). Numai atunci activează semnalizarea acustică (buzzer), avertizând clar utilizatorii. Toate aceste procese se desfășoară automat, fără intervenția directă a utilizatorului, iar sistemul revine în modul de supraveghere după dispariția oricărui pericol.

2.2 Schema bloc a sistemului

Schema bloc a sistemului ilustrează modul în care principalele componente sunt conectate și colaborează:

- Senzor accelerometru – detectează vibrațiile și transmite un semnal electronic.
- Bloc CLC – procesează semnalul de vibrație în funcție de semnal.
- Bloc CLS – interpretează rezultatul și generează semnalul de alertă.
- LED-uri și buzzer – semnalizează vizual și acustic nivelul de pericol.
- Săgeata de revenire – arată că sistemul se resetează și revine în starea de stand-by după dispariția pericolului.

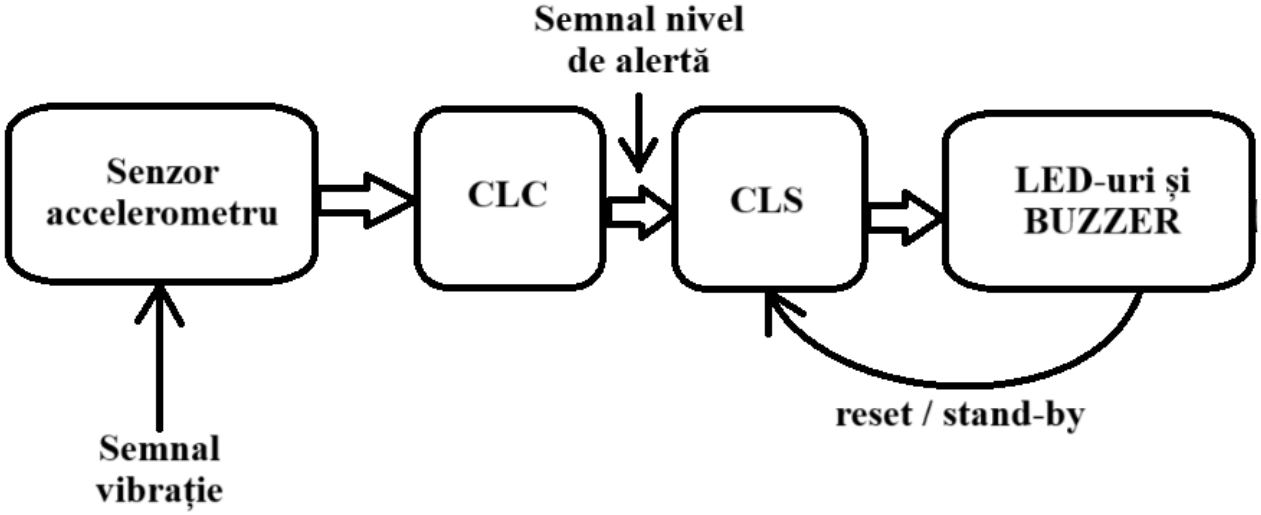

Figura 2.2.1 – Schema bloc a sistemului automat de monitorizare și avertizare pentru cutremure

Capitolul 3. Proiectarea și implementarea sistemului

În acest capitol sunt prezentate funcționarea și logica fiecărui circuit: CLC, CLS și PS. Pentru acestea sunt descrise tabela de adevăr, măștile folosite, graful de stări și algoritmii principali.

CLC interpretează semnalul de la senzorul accelerometru și stabilește dacă există risc de cutremur. Dacă este cazul, CLS verifică dacă vibrațiile periculoase persistă suficient de mult, pentru a evita alarmele false. La nevoie, PS comandă activarea LED-urilor și a buzzer-ului și gestionează revenirea sistemului la starea de stand-by.

3.1 Circuitul logic combinațional (CLC)

Acest circuit este folosit pentru a interpreta datele primite de la senzorul accelerometru. Dacă unul dintre switch-urile de intrare semnalează un anumit nivel (scăzut, mediu sau ridicat), CLC aprinde LED-ul corespunzător pentru a indica utilizatorului situația curentă, în plus pentru un nivel ridicat al vibrațiilor, se va porni un BUZZER, în cazul nostru un LED asociat acestuia pentru avertizare. Astfel, pentru fiecare nivel de vibrație detectat, sistemul reacționează imediat.

Dacă niciun switch nu este activat sau dacă valorile de la intrare nu corespund unui nivel valid, sistemul rămâne într-o stare de așteptare (IDLE), fără a semnaliza o stare de pericol, până când apare o nouă selecție la intrare.

Astfel, CLC-ul va avea 3 intrări și 4 ieșiri aferente situației prezentate anterior.

Intrări

SW 7	SW 6	SW 5	SW 4	SW 3	SW 2	SW 1	SW 0
-	-	-	-		VIB_R	VIB_M	VIB_S

Tabel 3.1.1 – intrări CLC

Ieșiri

LED 7	LED 6	LED 5	LED 4	LED 3	LED 2	LED 1	LED 0
-	-	-	-	BUZZER	NIV_R	NIV_M	NIV_S

Tabel 3.1.2 – ieșiri CLC

SW2-SW0 reprezintă senzori activi în '1', astfel:
VIB_R – vibrație ridicată
VIB_M – vibrație medie
VIB_S – vibrație scăzută

Sistemul este automat, astfel nu este nevoie de programare a vreunui mod de operare.

CLC-ul va avea ca intrări senzorul cu cele 3 nivele, astfel că SW2-SW0 vor fi considerate cele trei intrări ale CLC-ului.

Măștile specifice fiecărui nivel în parte sunt calculate prin operații logice, atribuind fiecărei măști valoarea 1, shift-ată ulterior de n (n – numărul de poziții necesare shift-ării, astfel încât bitul ajunge pe aceeași poziție cu intrarea senzorului în cauză) ori la stânga.

Intrare	VIB_R	VIB_M	VIB_S
Mască_VIB_R	1	0	0

Tabel 3.1.3 – Mască VIB_R

Mască_VIB_R = 0x04 → rezultat în urma shift-ării (1 << 2) ⇒ VIB_R = IN & 0x04;

Intrare	VIB_R	VIB_M	VIB_S
Mască_VIB_M	0	1	0

Tabel 3.1.4 – Mască VIB_M

Mască_VIB_R = 0x02 → rezultat în urma shift-ării (1 << 1) ⇒ VIB_M = IN & 0x02;

Intrare	VIB_R	VIB_M	VIB_S
Mască_VIB_R	0	0	1

Tabel 3.1.5 – Mască VIB_S

Mască_VIB_S = 0x01 → (1) ⇒ VIB_S = IN & 0x01;

În urma aflării măștii, se calculează câte un rezultat intermediat pentru fiecare bit de intrare, prin efectuarea unei operaţii logice AND între registrul de intrare pe care sunt poziționați biții IN și masca obținută anterior.

Intrare	VIB_R	VIB_M	VIB_S
TMP_VIB_R	VIB_R	0	0
TMP_VIB_M	0	VIB_M	0
TMP_VIB_S	0	0	VIB_S

Tabel 3.1.6 – Rezultatul shift-ării

Indexul aferent fiecărui semnal este obținut prin efectuarea operației logice SAU între rezultate.

INDEX = TMP_VIB_R | TMP_VIB_M | TMP_VIB_S ;

INDEX	VIB_R	VIB_M	VIB_S

Tabel 3.1.7 – INDEX

Intrarea CLC-ului, având trei biţi, poate fi descrisă prin 8 valori, însă semnalele pot fi doar active individual, astfel rezultă tabelul:

VIB_R	VIB_M	VIB_S
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

Și ținând cont că senzorul este activ în 1 ⇒

VIB_R	VIB_M	VIB_S	Semnificație
0	0	0	-
0	0	1	Activ VIB_S
0	1	0	Activ VIB_M
0	1	1	-
1	0	0	Activ VIB_R
1	0	1	-
1	1	0	-
1	1	1	-

Tabel 3.1.8 – Semnificație combinații posibile de biți

Astfel, pe baza semnificaţiei celor 8 combinaţii de valori ale biţilor de intrare, se pot determina ieşirile, astfel:

INTRARE				IEȘIRE					
VIB_R	**VIB_M**	**VIB_S**	**Semnificație**	**LED 3**	**LED 2**	**LED 1**	**LED 0**	**Semnificație**	**Val. Hexa**
0	0	0	-	0	0	0	0	Stare IDLE	0x00
0	0	1	Activ VIB_S	0	0	0	1	Nivel scăzut	0x01
0	1	0	Activ VIB_M	0	0	1	0	Nivel mediu	0x02
0	1	1	-	0	0	0	0	Stare IDLE	0x00
1	0	0	Activ VIB_R	1	1	0	0	Nivel ridicat (Alertă)	0x0C
1	0	1	-	0	0	0	0	Stare IDLE	0x00
1	1	0	-	0	0	0	0	Stare IDLE	0x00
1	1	1	-	0	0	0	0	Stare IDLE	0x00

Tabel 3.1.9 – Intrări / Ieșiri CLC și semnificații

Tabela de adevăr a CLC-ului va fi definită ca vector TAB cu 8 elemente, după cum urmează:

```
char TAB[8] = {0x00, 0x01, 0x02, 0x00, 0x0C, 0x00, 0x00, 0x00}
```

Codul asociat funcției CLC

```
char in;
char monitorizare;
char TAB[] = {0x00, 0x01, 0x02, 0x00, 0x0C, 0x00, 0x00, 0x00}

void CLC(void) {
        char VIB_R, VIB_M, VIB_S, selectie;
        VIB_R = in & 0x04;
        VIB_M = in & 0x02;
        VIB_S = in & 0x01;
        selectie = VIB_R | VIB_M | VIB_S;
        monitorizare = TAB[selectie];
}
```

3.2 Circuitul logic secvențial (CLS)

Acest circuit este responsabil de gestionarea întregului proces de monitorizare și avertizare a cutremurelor, bazându-se pe diferite stări. CLS primește permanent date de la CLC despre nivelul de vibrație detectat și parcurge, în funcție de situație, stări predefinite.

În acest capitol este prezentat modul în care funcționează circuitul logic secvențial, modelat printr-un automat finit de stări (FSM), care decide comportamentul sistemului atât în funcție de starea curentă, cât și de valorile citite de la senzori.

Conform cerinței, automatul de stare finit pentru gestionarea alertelor are patru stări:

1. Monitorizare (citirea constantă a senzorului)

2. Avertizare (dacă vibrațiile sunt moderate, activare LED de avertizare)

3. Alarmă activă (dacă vibrațiile persistă, activare sirenă și afișaj „Cutremur detectat!”)

4. Reset automat (dacă vibrațiile scad sub pragul critic)

Pentru crearea grafului asociat CLS, este necesară definirea stărilor, respectiv modul de tranziţie dintr-o stare în alta.

Adițional, am adăugat o stare intermediară (S2*), între Avertizare și Alarmă activă care este esențială pentru funcționarea corectă a sistemului.

Această stare intermediară permite sistemului să „numere” timpul în care nivelul ridicat de vibrații este menținut, astfel încât alarma să fie declanșată doar dacă pericolul persistă mai mult de 3 secunde consecutive. Fără această stare, alarma s-ar activa imediat ce apare nivelul ridicat, ceea ce ar duce la multe alarme false pentru vibrații de scurtă durată, lucru pe care dorim să-l evităm, conform cerinței.

Prin urmare, FSM-ul va parcurge pe rând stările de monitorizare, avertizare, numărătoare (persistență), alarmă activă și reset automat, asigurând o funcționalitate cât mai apropiată de cea reală.

Stare	Adresa tabelei de semnale relevante	Semnificație
1	A0	Monitorizare
2	A1	Avertizare
2*	A2	Numărare (persistență)
3	A3	Alarmă activă
4	A4	Reset automat

Tabel 3.2.1 – Adresele asociate stărilor

Astfel, graful asociat CLS este:

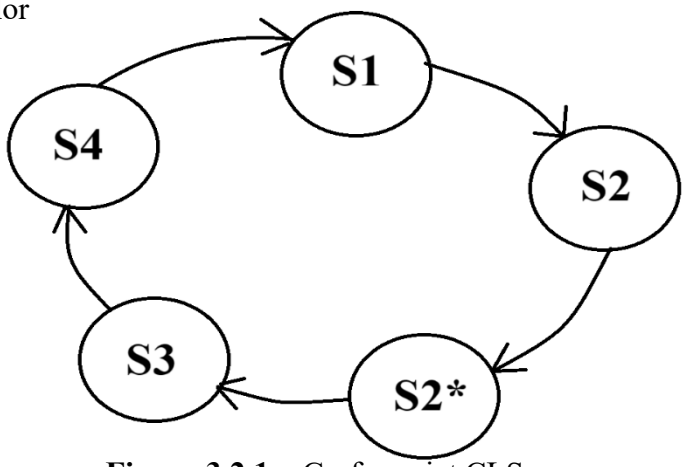

Figura 3.2.1 – Graf asociat CLS

Starea curentă		Intrări	Starea următoare		Hexa
S[index]	Codificare	CLK	S'[index]	Codificare	
S1	000	1	S1	000	0x00
S1	000	0	S2	001	0x01
S2	001	1	S2	001	0x01
S2	001	0	S2*	010	0x02
S2*	010	1	S2*	010	0x02
S2*	010	0	S3	011	0x03
S3	011	1	S3	011	0x03
S3	011	0	S4	100	0x04
S4	100	1	S4	100	0x04
S4	100	0	S1	000	0x00

Tabel 3.2.2 – Tabel de tranziție al stărilor

Conform detaliilor prezentate anterior, cele cinci tabele de semnale relevante asociate CLS pot fi descrise astfel:

$A0$[ ] = {0x00, 1, 'T', 0}

$A1$[ ] = {0x00, 2, 'T', 1}

$A2$[ ] = {0x00, 3, 'T', 2}

$A3$[ ] = {0x00, 4, 'T', 3}

$A4$[ ] = {0x00, 0, 'T', 4}

Cod secvențial CLS

```
#define T 0x10    // terminator
#define NR 5       // număr stări

char Q = 0;                    // stare curentă
char in;
char out;

char *TABA[NR];            // vector adrese tabele

//tabele de semnale relevante
char A0[] = {0x00, 1, 'T', 0}; // Monitorizare
char A1[] = {0x00, 2, 'T', 1}; // Avertizare
char A2[] = {0x00, 3, 'T', 2}; // Numărătoare
char A3[] = {0x00, 4, 'T', 3}; // Alarmă activă
char A4[] = {0x00, 0, 'T', 4}; // Reset automat
```

```
// ieșiri pentru fiecare stare
char Tout[NR] = {0x01, 0x02, 0x00, 0x0C, 0x01};

TABA[0] = A0;
TABA[1] = A1;
TABA[2] = A2;
TABA[3] = A3;
TABA[4] = A4;

void cls(void) {
      char i;
      char *adr;
      char ready;
      adr = TABA[Q];
      i = 0;
      ready = 0;

      while (!ready) {
         if (in == *(adr+i)) {
             Q = *(adr+i+1);
             ready = 1;
         }
         else if (*(adr+i)==T) {
             ready = 1;
         }
         else {
             i = i + 2;
         }
    }
    out = Tout[Q];
}
```

3.3 Procesul secvențial (PS) – FSM-ul general

Funcționarea modulului software al sistemului de monitorizare și avertizare pentru cutremure se bazează pe un proces secvențial responsabil pentru parcurgerea automată a tuturor stărilor definite de automatul finit (FSM) și pentru gestionarea corectă a semnalizării vizuale și acustice în funcție de nivelul de vibrație detectat. Acest proces se bazează pe datele de intrare primite de la circuitul logic combinațional (CLC) și pe condițiile temporale gestionate de circuitul logic secvențial (CLS).

Procesul secvențial asigură ca fiecare stare a sistemului să fie parcursă corect, iar trecerea de la o stare la alta să se facă doar când sunt îndeplinite condițiile din cerință. Astfel, se evită alarmele false și utilizatorul primește mereu o reacție corectă din partea sistemului, indiferent de tipul sau durata vibrațiilor detectate.

Astfel, conform cerinței, programarea sistemului de monitorizare și avertizare se face astfel:

Sistemul pornește din starea de monitorizare S1, unde se apelează circuitul combinațional CLC pentru a interpreta valorile citite de la senzor. În funcție de rezultatul procesării logice, CLC poate genera trei tipuri de ieșiri relevante: 0x01 (vibrație scăzută), 0x02 (vibrație medie) sau 0x0C (vibrație ridicată). Fiecare din aceste ieșiri determină parcursul sistemului în etapele următoare.

Dacă este detectată o vibrație scăzută (0x01), sistemul trece în starea S'5, unde LED-ul verde (LED0) se aprinde, semnalizând o stare de normalitate și lipsa oricărei alerte. Imediat după aceea se apelează circuitul secvențial CLS pentru a valida starea și a reveni în starea de reset automat (S4), asigurând astfel un ciclu continuu de monitorizare și repornire a sistemului.

În cazul în care este detectată o vibrație medie (0x02), se intră în starea S'6, unde LED-ul verde (LED0) este stins. Din această stare, sistemul trece automat în starea S2, unde LED-ul portocaliu (LED1) este activat pentru a avertiza utilizatorul asupra unei posibile probleme. După semnalizarea vizuală de avertizare, CLS este apelat pentru a verifica dacă vibrațiile persistă sau nu, iar apoi sistemul revine în starea de reset automat (S4) și, ulterior, în monitorizare.

Dacă sistemul identifică o vibrație ridicată (0x0C), se activează starea S'7, în care atât LED-ul verde (LED0), cât și LED-ul portocaliu (LED1) se asigură că sunt stinse. Din această stare, sistemul avansează în S2*, starea de persistență, unde începe numărarea timpului pentru verificarea duratei semnalului de vibrație ridicată. Dacă vibrația ridicată nu persistă timp de 3 secunde (contorul nu ajunge la pragul necesar), sistemul trece în S'9, apelând CLS și ajungând în starea de reset (S4), evitând astfel declanșarea unei alarme false. În schimb, dacă vibrația ridicată persistă 3 secunde sau mai mult, sistemul avansează în S'8, apoi către starea de alarmă activă (S3).

În S3, sistemul aprinde intermitent LED-ul roșu (LED2) și pornește buzzer-ul (LED3), semnalizând clar detectarea unui cutremur sau a unei alarme reale. Cât timp vibrațiile persistă, sistemul rămâne în această stare de alertă. Odată ce vibrațiile dispar, sistemul tranzitează către starea de reset automat (S4), unde toate LED-urile (LED0, LED1, LED2, LED3) se sting, semnalizând resetarea și pregătirea pentru un nou ciclu de monitorizare.

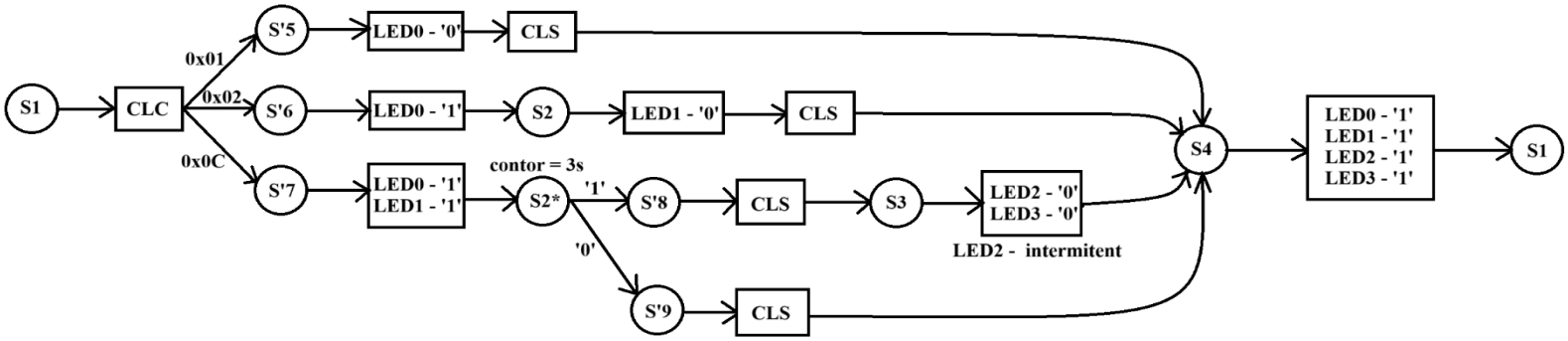

Figura 3.3.1 – Graful de tranziții PS

3.4 Codul complet a modulului software (CLC + CLS + PS)

```
#define T 0x10    // terminator
#define NR 5      // număr stări

char Q = 0;                     // stare curentă
char in;                   // intrare sistem (PIND)
char out;                  // ieșire finală (PORTB)
char S;                    // stare PS

char TAB[] = {0x00, 0x01, 0x02, 0x00, 0x0C, 0x00, 0x00, 0x00};

char *TABA[NR];            // vector adrese tabele

//tabele de semnale relevante
char A0[] = {0x00, 1, T, 0}; // Monitorizare
char A1[] = {0x00, 2, T, 1}; // Avertizare
char A2[] = {0x00, 3, T, 2}; // Numărătoare
char A3[] = {0x00, 4, T, 3}; // Alarmă activă
char A4[] = {0x00, 0, T, 4}; // Reset automat

// ieșiri pentru fiecare stare
char Tout[NR] = {0x01, 0x02, 0x00, 0x0C, 0x01};

char monitorizare = 0;

// CLC
void clc(void) {
          char VIB_R, VIB_M, VIB_S, selectie;
          VIB_R = in & 0x04;
          VIB_M = in & 0x02;
          VIB_S = in & 0x01;
          selectie = VIB_R | VIB_M | VIB_S;
          monitorizare = TAB[selectie];
}
```

```
char out_cls = 0;

//CLS
void cls(void) {
      char i;
      char *adr;
      char ready;
      adr = TABA[Q];
      i = 0;
      ready = 0;

      while (!ready) {
         if (in == *(adr+i)) {
             Q = *(adr+i+1);
             ready = 1;
         }
         else if (*(adr+i)==T) {
             ready = 1;
         }
         else {
             i = i + 2;
         }
    }
    out_cls = Tout[Q];
}

char clock1 = 0;
int contor = 0;

//functiile de intreruperi
interrupt [TIM0_OVF] void timer0_ovf_isr(void)
{
      TCNT0=0x3C;
}
```

```
void intrerupere1(void){
      contor = (contor + 1) % 150; //intreruperea pentru pornire alerta
            if(contor == 1){
                  clock1 = (clock1 + 1) % 2;
            }
            if (clock1 == 1){
                  out = 0x0C;
            }
                  else Q = 4;

}



int k = 0;
int clock2 = 0;

void intrerupere2(void){
      k = (k + 1) % 25; //intreruperea pentru ledul rosu intermitent
            if(k == 0){
             clock2 = (clock2 + 1) % 2; /*clock de 0.5 secunde*/
            }
            while (Q == 5){ // Am înlocuit S2* cu S5
                  if (clock2 == 1){
                        out = 0x0C;
                  }
                  else out = 0x08;
            }
}

void main(void)
{
    // Inițializări tabel FSM
    TABA[0] = A0;
    TABA[1] = A1;
    TABA[2] = A2;
```

```
    TABA[3] = A3;
    TABA[4] = A4;
    Q = 0;        // starea CLS inițială
    S = 1;        // starea PS inițială (monitorizare)
    out = 0x00;   // ieșire inițială

#asm("sei")

    while (1)
    {
        in = PIND;      // citește intrarea de la senzori (sau switch-uri)
        clc();           // actualizează variabila monitorizare

        switch (S)
        {
            case 1: // S1 – Monitorizare
                out = 0x01;
                if (monitorizare == 0x01)
                {
                    S = 2; // trece la S'5 - vibrație scăzută
                }
                else if (monitorizare == 0x02)
                {
                    S = 3; // trece la S'6 - vibrație medie
                }
                else if (monitorizare == 0x0C)
                {
                    S = 4; // trece la S'7 - vibrație ridicată
                }
                break;

            case 2: // S'5
                out = 0x01; // LED0 ON
                cls();
                S = 9;      // reset automat
```

```
    break;

case 3: // S'6
    out = 0x02;     // LED0 OFF, LED1 ON
    cls();
    S = 9;
    break;

case 4: // S'7
    out = 0x00;
    S = 5;
    break;

case 5: // S2* - măsurare persistență
    out = 0x00;
    intrerupere1();
    if(out == 0x0C){
          S = 6; // trece la starea de alarmă
    }
    else {
          S = 7;
    }
    break;

case 6: // S8 - persistență detectată
    cls();
    S = 8;
    break;

case 7: // S9 - reset din persistență eșuată
    cls();
    S = 9;
    break;

case 8: // S3 - stare de alarmă
```

```
                out = 0x0C;
                intrerupere2();
                cls();
                S = 9;
                break;

            case 9: // S4 - reset automat
                out = 0x00; // LED0-3 OFF
                S = 1;      // revine în monitorizare
                break;

            default:
                S = 1;
                break;
        }
        PORTB = out;
    }
}
```

Capitolul 5. Testarea și validarea sistemului

Pentru a verifica corectitudinea funcționării sistemului automat de monitorizare și avertizare pentru cutremure, au fost realizate mai multe teste funcționale folosind diferite combinații de intrare provenite de la senzori. Codurile testate corespund valorilor binare generate de nivelurile de vibrații și sunt analizate de circuitul logic combinațional. Rezultatul influențează comportamentul sistemului în starea de comparare și declanșează acțiuni în direcționare.

5.1 Metodologia de testare

Sistemul este testat după anumite etape de testare, iar acestea sunt:

1. Scrierea și compilarea codului în CodeVisionAVR

    - Scriu codul C complet în CodeVisionAVR (inclusiv FSM, citirea porturilor, logica de sortare și afișare).

2. Importul proiectului în AVR Studio pentru debugging

    - Creez un nou proiect în AVR Studio, selectând aceeași placă (ATmega164P).

3. Setarea punctelor de întrerupere (breakpoints)

    În AVR Studio, plasez breakpoints pe liniile cheie:

    - la citirea portului PIND (pentru urmărirea codului de intrare)
    - în blocul switch(S) (pentru a verifica ce cale logică este urmată)
    - la liniile ce scriu în PORTB (pentru observarea ieșirilor)

4. Simularea intrărilor

    În I/O View, setez manual valorile pentru PIND pentru a simula codurile dorite:

    - PIND = 0x01 (0001) → test pentru vibrații scăzute
    - PIND = 0x02 (0010) → test pentru vibrații medii
    - PIND = 0x04 (0100) → test pentru vibrații ridicate

5. Rularea pas cu pas (Step Into / Step Over)

Rulez programul în modul pas cu pas pentru a observa cum trece automatul dintr-o stare în alta.

Verific în timp real dacă:

- starea FSM se actualizează corect
- codul este interpretat corect
- se activează porturile de ieșire (LED-uri, buzzer) în funcție de caz

5.2 Rezultate obținute și interpretare

Figuri 5.2 – Testare funcționare

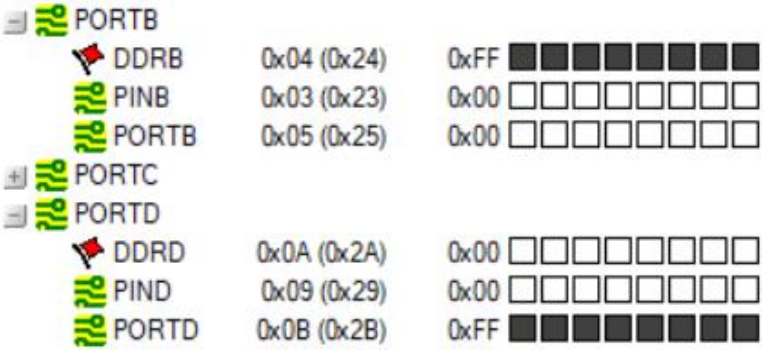

Fig. 5.2.1 – Starea de monitorizare a sistemului

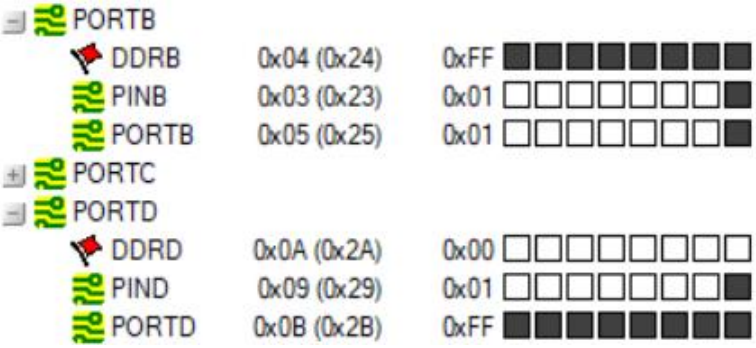

Fig. 5.2.2 – Testare nivel scăzut de vibrații

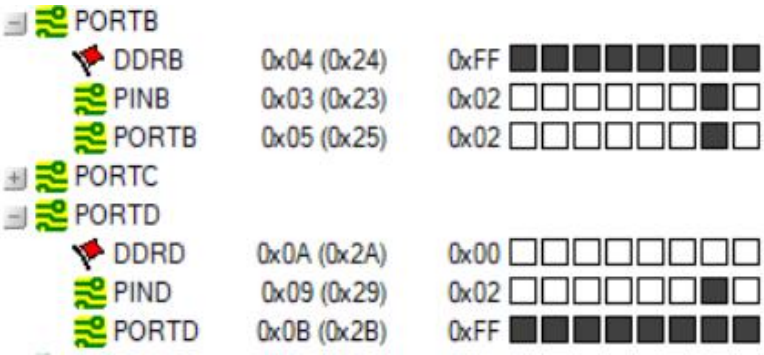

Fig. 5.2.3 – Testare nivel mediu de vibrații

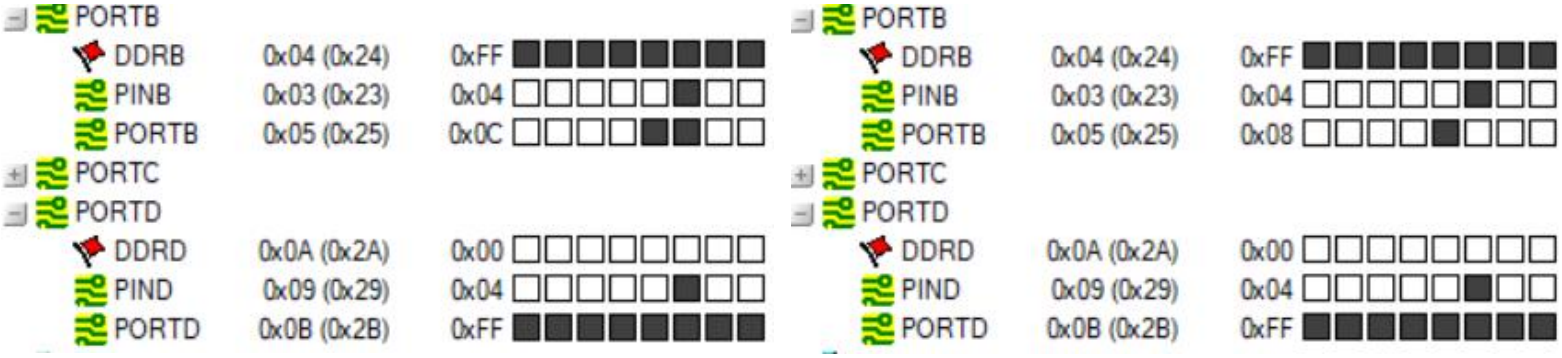

Fig. 5.2.4 – Testare nivel ridicat de vibrații (LED roșu ON - intermitent + Buzzer ON)

Figurile prezentate justifică funcționalitatea codului sursă, pentru fiecare intrare rezultând ieșirea corespunzătoare (Fig. 5.2.1 – intrarea: 0x00/ieșirea: 0x00, Fig. 5.2.2 – intrarea: 0x01/ieșirea: 0x01, Fig. 5.2.3 – intrarea: 0x02/ieșirea: 0x02, Fig. 5.2.4 – intrarea: 0x04/ieșirea: 0x0C sau ieșirea: 0x08 – pentru că LED-ul roșu este intermitent).

Concluzii și perspective

Prin realizarea acestui sistem automat de monitorizare și avertizare pentru cutremure, am demonstrat că se poate construi un dispozitiv electronic eficient, capabil să detecteze rapid și să semnalizeze utilizatorului prezența unor vibrații anormale, folosind componente digitale și o structură logică clară. Folosirea unui senzor de vibrații, combinată cu logica CLC (circuit logic combinațional) și CLS (circuit logic secvențial), permite sistemului să identifice corect nivelurile de pericol și să filtreze alarmele false, activând alarma doar atunci când vibrațiile periculoase persistă cel puțin 3 secunde.

Proiectul a acoperit atât proiectarea hardware (alegerea componentelor și a structurii circuitului), cât și dezvoltarea software (implementarea FSM, programarea alertelor, testarea și validarea funcționalității). Testarea a confirmat că sistemul reacționează corect la orice nivel de vibrații simulate, iar semnalizarea vizuală și acustică funcționează conform cerințelor. LED-urile indică în timp real starea sistemului (verde - normal, galben - atenționare, roșu intermitent - alarmă), iar buzzerul asigură avertizarea sonoră la pericol real.

În concluzie, acest sistem reprezintă un exemplu clar de aplicare a logicii digitale în detecția automată a riscurilor din mediul ambiant, putând fi extins sau adaptat pentru alte aplicații de siguranță. Prin filtrarea automată a alarmelor false și revenirea rapidă la monitorizare după dispariția pericolului, sistemul asigură atât protecție, cât și fiabilitate în funcționare.